Proxy

Proxy · Lazy-load heavy image assets · Build with Waqar

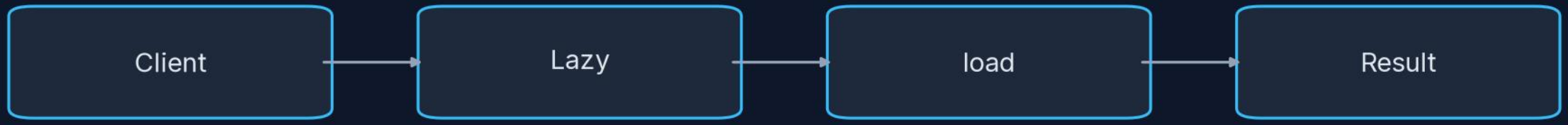
Today's Hook

- Problem: Lazy-load heavy image assets
- Pattern: Proxy — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Lazy-load heavy image assets
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Proxy: Lazy-load heavy image assets

Real-World Scenario

- Running example: Lazy-load heavy image assets
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.gof.structural.proxy;
import com.buildwithwaqar.patterns.gof.structural.proxy.LazyImageProxy;
/** Problem: gallery page must not load every image from disk until displayed. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Proxy problem: lazy image load ===");
    var img = new LazyImageProxy("/cdn/hero.png");
    System.out.println("loadedBeforeScroll=" + img.loaded());
    System.out.println(img.display());
    System.out.println("loadedAfter=" + img.loaded());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.proxy;

interface Image {
    String display();
}

final class RealImage implements Image {
    private final String path;
    RealImage(String path) { this.path = path; }
    public String display() { return "showing:" + path; }
}

/** Virtual proxy — lazy-loads the real subject. */
public final class LazyImageProxy implements Image {
    private final String path;
    private RealImage real;

    public LazyImageProxy(String path) { this.path = path; }

    @Override
    public synchronized String display() {
        if (real == null) real = new RealImage(path);
        return real.display();
    }
```

Solution (2)

```
    public boolean loaded() { return real != null; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.proxy;
import com.buildwithwaqar.patterns.gof.structural.proxy.LazyImageProxy;
/** Problem: gallery page must not load every image from disk until displayed. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Proxy problem: lazy image load ===");
    var img = new LazyImageProxy("/cdn/hero.png");
    System.out.println("loadedBeforeScroll=" + img.loaded());
    System.out.println(img.display());
    System.out.println("loadedAfter=" + img.loaded());
  }
}
```

Walkthrough Output

```
=== Proxy problem: lazy image load ===
loadedBeforeScroll=false
showing:/cdn/hero.png
loadedAfter=true
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Proxy.

Related Patterns

- Contrast Proxy with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Proxy → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.